

CHAPTER-1

INTRODUCTION

This internship report summarizes my experience in the "FPGA BASED VLSI DESIGN " summer internship program at Sense Semiconductor & IT Solutions Pvt. Ltd.(SSIT) in 2024. The program provided an in-depth exploration of FPGA-based digital system design, covering a broad range of topics from Verilog constructs to advanced memory systems. The following report outlines each module covered during the internship and the practical knowledge gained through hands-on projects and training.

1.1 NEXYS A7:

The Nexys A7 is an FPGA development board by Digilent that features an AMD Artix-7 FPGA and is designed for teaching and prototyping digital circuits. It includes various onboard components and I/O for a wide range of projects, such as sensors, memory, and networking capabilities, making it suitable for applications from simple combinational circuits to complex embedded systems.

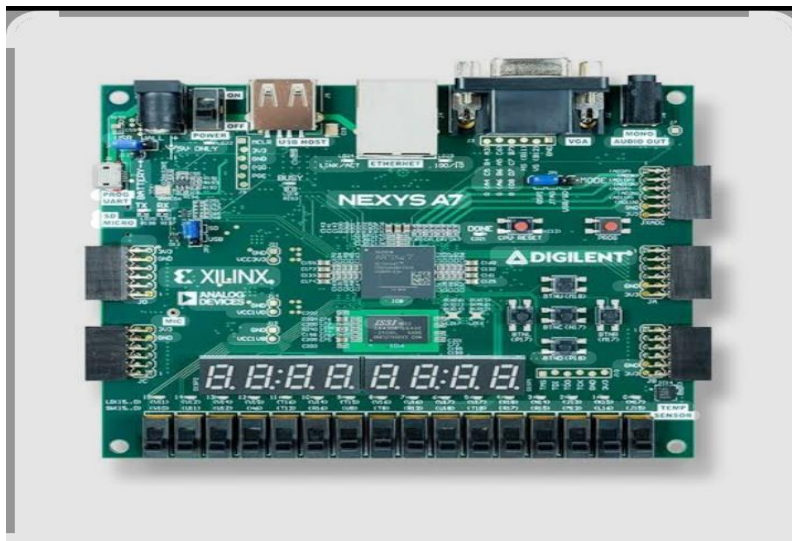


Fig. 1: Nexys A7

Key Features:

1. **Xilinx Artix-7 FPGA (XC7A100T-1CSG324C)**
 - 101,440 logic cells
 - 4,860 Kb block RAM
 - 240 DSP slices for advanced signal processing operations
 - 6.6 Gb/s high-speed transceivers for fast data transfer

2. On-board Clock

- 100 MHz clock for accurate and stable timing in FPGA designs.

3. Inputs/Outputs:

- 16 user switches, 16 user LEDs, and 5 push buttons.
- 4-digit seven-segment display for numerical output.
- 12-bit VGA output port for video display.
- Pmod connectors to connecting additional external peripherals.
- 12-bit analog-to-digital converter (ADC) for reading analog signals.

4. Communication Interfaces:

- USB-UART bridge for serial communication with a PC.
- USB-JTAG for programming and debugging the FPGA.
- SPI, I²C communication protocols supported through Pmod interfaces.
- Ethernet port for network-based applications.

5. Power Supply:

- Can be powered via USB or external power adapter.
- Provides flexible and reliable power options for portable and lab applications.

6. Programming and Development Tools:

- Compatible with Xilinx Vivado Design Suite (WebPACK Edition, free).
- Supports VHDL and Verilog for hardware design.
- On-board Quad-SPI Flash memory for configuration storage.

Applications:

- Students can design and test **custom processors (CPU), ALUs, register files, and memory controllers.**
- Used to implement and verify new **digital system architectures, encryption algorithms, image/video processing systems, or AI accelerators.**
- The Artix-7 FPGA supports research into **power-efficient FPGA designs.**

1.2 Xilinx Vivado Software

Vivado Design Suite is a software suite for synthesis and analysis of hardware description language (HDL) designs, superseding Xilinx ISE with additional features for System on Chip development and high-level synthesis. Vivado represents a ground-up rewrite and re-thinking of the entire design flow. Vivado is compatible on Windows OS, LINUX, MAC OS. Xilinx Vivado is a software suite for designing digital logic and implementing FPGAs. It has many features, including:

- Design flows Vivado has a platform board flow that allows users to select an existing Xilinx evaluation platform board as a design target. This flow exposes the IP interfaces on the board, which can be quickly configured and selected.
- I/O and clock planning Vivado has tools for automatic and interactive I/O placement. It also has an I/O Planning view layout that shows the relationship between the device package's physical pins and the I/O banks' die pads.
- High-level synthesis Vivado has a toolchain that can convert C code into programmable logic.
- Design checkpoints Vivado uses a physical design database called design checkpoints to store placement and routing. These checkpoints can be saved and restored at key points in the design flow.
- Reporting Vivado can generate reports on power consumption, design rule violations, and final timing. These reports can be output to files or directed to the Vivado IDE.

CHAPTER-2

TECHNOLOGIES COVERED

2.1 FPGA ARCHITECTURE AND DESIGN FLOW:

This module provided a comprehensive overview of FPGA architecture and the design flow:

- **FPGA Components:** Detailed study of CLBs, IOBs, and interconnects.
- **Design Flow:** Steps from design entry to programming and testing on FPGA.

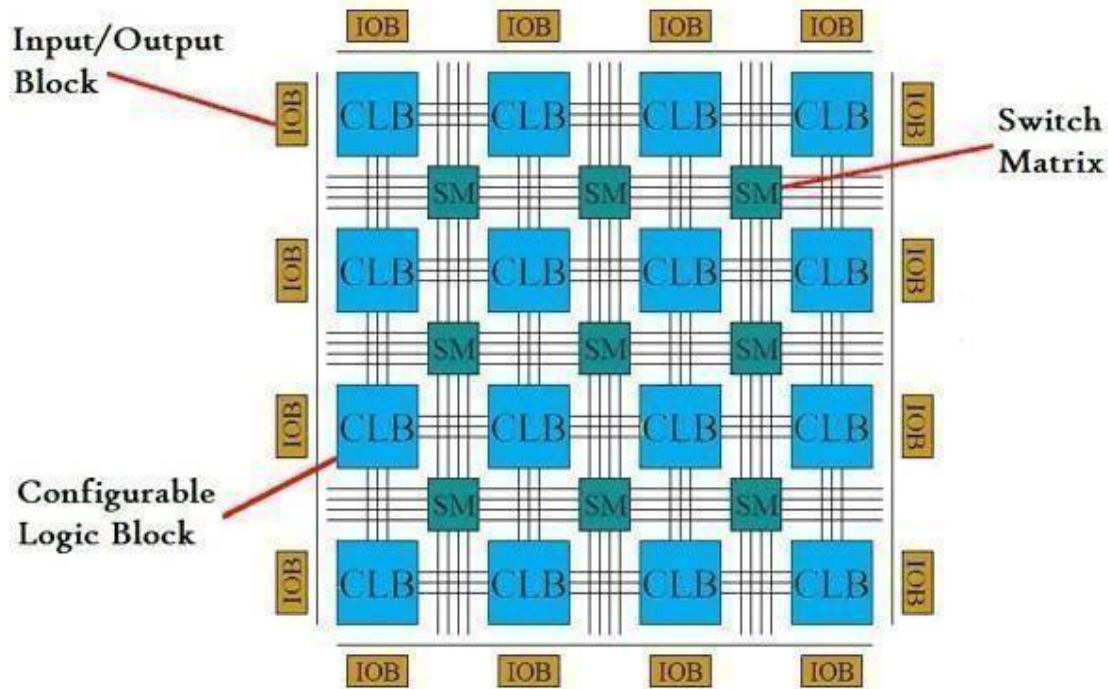


Fig. 1: FPGA Architecture

FPGA COMPONENTS:

- **Configurable Logic Blocks (CLBs):** These are the fundamental building blocks of an FPGA. CLBs contain lookup tables (LUTs), flip-flops, and multiplexers that can be configured to perform various logical operations.

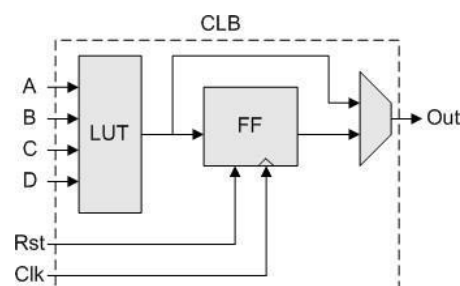


Fig. 2: Configurable Logic Blocks

- **Input/Output Blocks (IOBs):** These are the fundamental building blocks of an FPGA. CLBs contain lookup tables (LUTs), flip-flops, and multiplexers that can be configured to perform various logical operations.
- **Interconnects:** Interconnects are the routing channels that connect CLBs and IOBs, allowing them to communicate with each other.

DESIGN FLOW:

- **Design Entry:** This is the first step where the design is created using a hardware description language (HDL) like VHDL or Verilog.
- **Mapping:** The netlist is mapped onto the FPGA's CLBs, IOBs, and interconnects.
- **Placement:** The mapped components are placed onto the FPGA's resources, taking into account factors like timing and routing.
- **Routing:** The placed components are connected using the interconnects.
- **Timing Analysis:** The design is analyzed for timing constraints, ensuring that it meets the required performance.
- **Programming:** The final design is converted into a bitstream, which is loaded onto the FPGA.
- **Testing:** The programmed FPGA is tested to ensure it functions as expected.

2.2 VERILOG CONSTRUCTION AND COMBINATION:

In this module, we were introduced to Verilog, a hardware description language used for designing digital systems. We covered basic constructs including:

Data Types: Wires, registers, and their usage in designing combinational logic.

- **Wires:** Used to connect outputs of one module to inputs of another.
- **Registers:** Used to store values and maintain state.

Operators: Logical, arithmetic, and relational operators used in Verilog.

- **Logical Operators:** AND (&), OR (|), NOT (~), XOR (^)
- **Arithmetic Operators:** +, -, *, /, %

- **Relational Operators:** ==, =, >, <, >=, <=

Design Units: Module, ports, and connections to describe digital circuits.

- **Modules:** Basic building blocks of Verilog design, representing digital circuits.
- **Ports:** Inputs and outputs of a module, used to connect to other modules.
- **Connections:** Wires and registers used to connect ports and modules.

Assignments, Control structures, functions and tasks and Timing controls

- **Assignments:** Continuous assignment (assign) and procedural assignment (=)
- **Control Structures:** if-else statements, case statements, loops (initial, always)

2.3 MULTIPLEXERS AND DEMULTIPLEXERS:

Multiplexers (MUX) and Demultiplexers (DEMUX) are fundamental components in digital design.

Here's a summary of the key points:

Multiplexers (MUX):

- **Implementation:** A MUX selects one output from multiple inputs based on a select signal.
- **Universal Logic:** MUX can implement any Boolean function, making them a universal logic component.
- **Usage:** MUX is used in digital systems for:
 - Data selection
 - Bus management
 - Logic implementation

Demultiplexers (DEMUX):

- **Design:** A DEMUX routes a single input to multiple outputs based on a select signal.
- **Function:** DEMUX is used to distribute a single signal to multiple destinations.
- **Usage:** DEMUX are used in digital systems for:
 - Data distribution

- Bus management
- Memory addressing

2.4 ENCODER AND DECODER:

Decoders and Encoders, essential components for data routing and transformation:

Decoder: A decoder is a digital circuit that converts a coded input signal into multiple output signals. It takes in a single input and produces multiple outputs, depending on the code.

- **Binary-to-Decimal Decoders:** Convert binary codes to decimal numbers.
- **Binary-to-BCD (Binary Coded Decimal) Decoders:** Convert binary codes to BCD numbers.

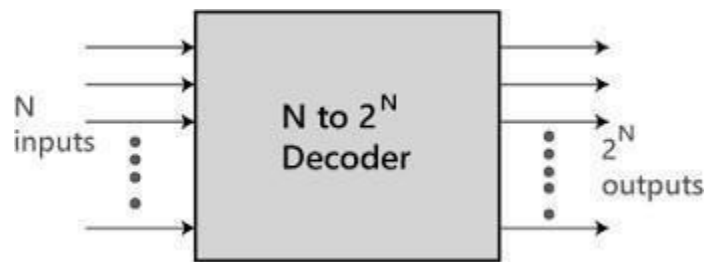


Fig. 3: Decoder Block Diagram

Fig. 5: DECODER

Encoder: An encoder is a digital circuit that converts 2^N input signals into a N output signals, typically using a binary code. It takes in multiple inputs and produces a coded output that represents the active input.

- **Priority Encoders:** Encode multiple inputs into a single output, prioritizing inputs based on their significance.
- **Binary Encoders:** Convert multiple inputs into a binary code, minimizing the number of outputs.

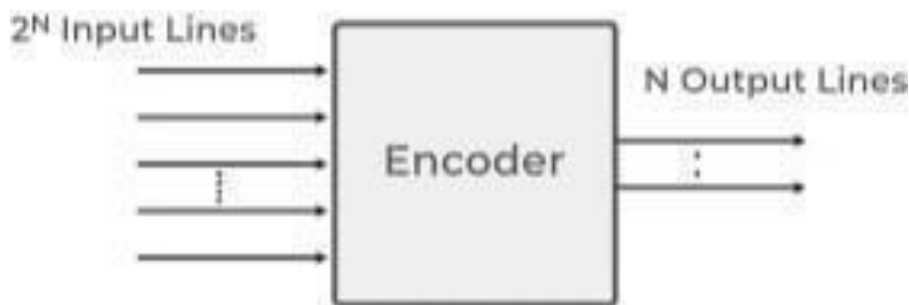


Fig. 4: Encoder

2.5 EVENT QUEUE AND DESIGN GUIDELINES:

This module introduced the concept of event-driven simulation in digital design:

Event Queue:

- **Event-driven simulation:** A simulation methodology where events (changes in signal values) are scheduled and processed in a queue.
- **Event queue operation:**
 - **Event scheduling:** Events are added to the queue based on their time of occurrence.
 - **Event processing:** Events are processed in the order they were scheduled, updating signal values and triggering new events.
- **Importance of event queue:**
 - Accurate modeling of digital circuit behavior
 - Efficient simulation of complex designs
- **Design Guidelines:**
- **Writing synthesizable Verilog code:**
 - Use standard Verilog syntax and semantics
 - Avoid using non-synthesizable constructs (e.g., initial blocks, function calls)
 - Use proper coding style and structure
- **Optimizing design performance:**
 - Minimize logic depth and complexity
 - Use efficient coding techniques (e.g., pipelining, retiming)
 - Optimize resource usage (e.g., reduce area, power consumption)
- **Best practices:**
 - Use meaningful naming conventions and comments
 - Test and verify designs thoroughly
 - Document design intent and functionality

2.6 DESIGN USING SYNTHESIZABLE CONSTRUCT:

Sequential design involves circuits where outputs depend on past inputs, not just current inputs:

Flip-Flops:

- **D Flip-Flop:**

- Stores data (0 or 1)
- Has two inputs: Clock (CLK) and Data (D)
- Has two outputs: Q (output) and Q' (complement of Q)
- When CLK rises, Q takes the value of D
- **Verilog implementation:** always @ (posedge clk) Q <= D;

- **T Flip-Flop:**

- Toggles its output (Q) with each clock pulse
- Has two inputs: Clock (CLK) and Toggle (T)
- Has two outputs: Q and Q'
- If T=1, Q toggles with each CLK rise

- **Verilog implementation:** always @ (posedge clk) Q <= T? ~Q: Q; ● **JK Flip-Flop:**

- Most versatile and widely used flip-flop
- Has three inputs: Clock (CLK), J, K and two outputs: Q, Q'
- Operation depends on J and K values:
 - J=0, K=0: No change
 - J=0, K=1: Reset (Q=0)
 - J=1, K=0: Set (Q=1)
 - J=1, K=1: Toggle

Verilog implementation: always @ (posedge clk) Q <= (J & ~K)? 1: (~J & K)? 0: Q;

- **R Flip-Flop:**

Simplest type of flip-flop

- Has two inputs: Set (S), Reset (R) and two outputs: Q, Q'
- Operation depends on S and R values:

- S=1, R=0: Set (Q=1)
- S=0, R=1: Reset (Q=0)
- S=1, R=1: Invalid state
- S=0, R=0: No Change

Verilog implementation: `always @ (posedge clk) Q <= (S & ~R)? 1: (~S & R)? 0: Q;`

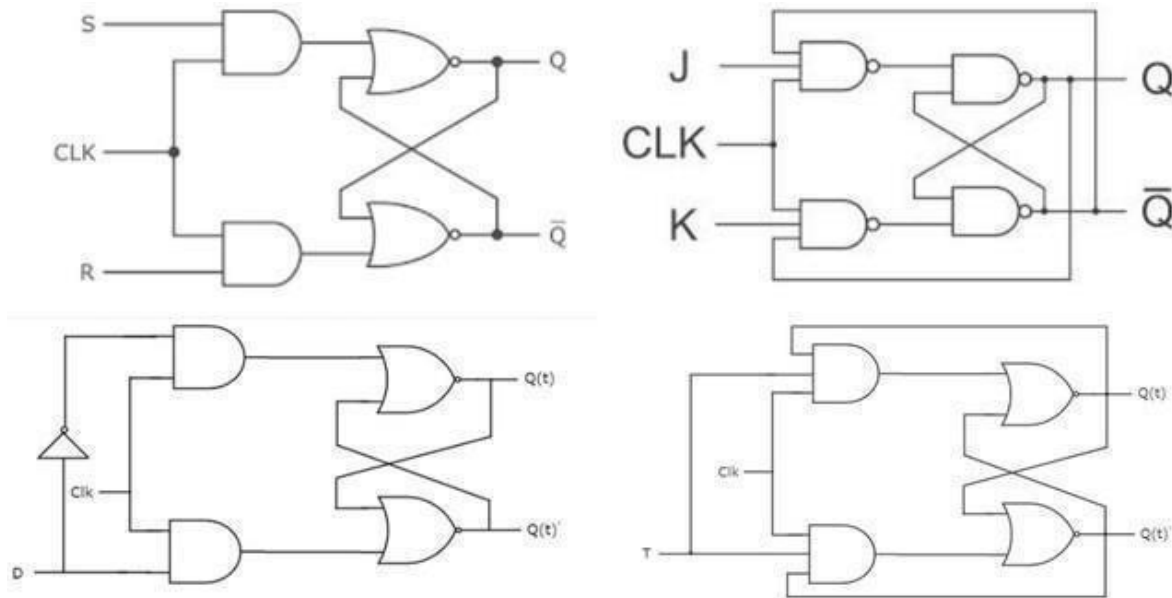


Fig. 5: Flip-Flops

Registers:

- A register is a group of flip-flops that store binary data.
- Each flip-flop represents one bit of data.
- Registers have multiple inputs and outputs, and can perform various operations like storing, shifting, and rotating data.

Types of registers:

- Parallel-in/Parallel-out (PIPO) register
- Parallel-in/Serial-out (PISO) register
- Serial-in/Parallel-out (SIPO) register
- Serial-in/Serial-out (SISO) register
- Shift register (stores and shifts data)

Counters:

- A counter is a digital circuit that counts the number of clock pulses or events.
- Counters can count up (increment) or down (decrement).

Types of counters:

- Asynchronous counter (ripple counter)
- Synchronous counter
- Up-down counter
- Ring counter
- Johnson counter

2.7 SEQUENTIAL, SNCHRONOUS AND ASYNCHRONOUS:

Counters, which are fundamental in timing and counting applications:

Synchronous Counters: A synchronous counter is a digital circuit where all flip-flops are triggered by the same clock edge, ensuring that all bits change simultaneously.

- All flip-flops are triggered by the same clock edge
- Clock signal is distributed to all flip-flops
- Counting sequence is predictable and controlled
- Design is simpler and more efficient
- **Example:** Binary counter, Decimal counter

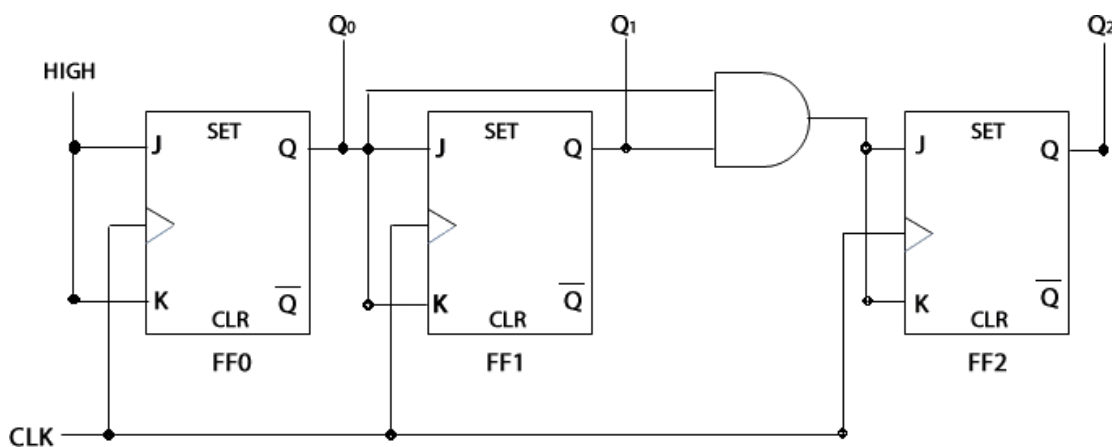


Fig. 6: Synchronous 3-Bit UP Counter

Asynchronous Counters: An asynchronous counter, also known as a ripple counter, is a digital circuit where each flip-flop is triggered by the output of the previous flip-flop, causing a ripple effect. Flip-flops

are triggered by different clock signals

- Each flip-flop has its own clock input
- Counting sequence is less predictable and more complex
- Design is more challenging and less efficient
- **Example:** Ripple counter, Up-down counter

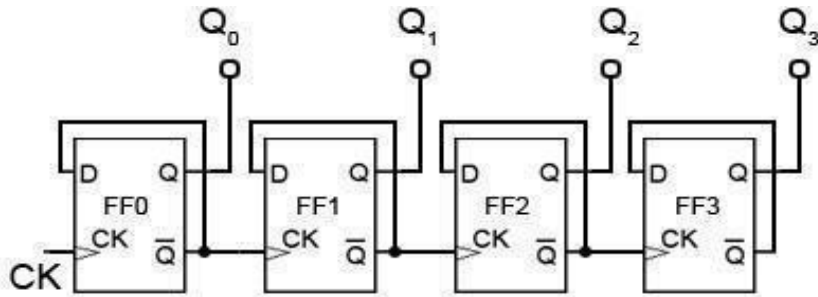


Fig. 7: Asynchronous 4-bit UP Counter Design Using

Synthesizable Constructs:

- Use Verilog's always blocks and edge-sensitive logic
- Define flip-flop inputs and outputs using synthesizable data types (e.g., reg, wire)
- Use clock signals to trigger flip-flops
- Implement counting logic using Verilog's arithmetic and logical operators

2.8 SHIFT REGISTERS AND FINITE STATE MACHINE(FSM):

Shift Registers: A shift register is a digital electronic circuit that stores and shifts binary data (0s and 1s) in a serial or parallel manner.

Functionality:

- Stores binary data in a sequence of flip-flops.
- Shifts data in a specified direction (left, right, or both).
- Can perform serial-to-parallel or parallel-to-serial conversions.

Types:

- Serial-In Parallel-Out (SIPO): Serial input, parallel output.
- Parallel-In Serial-Out (PISO): Parallel input, serial output.

- Serial-In Serial-Out (SISO): Serial input, serial output.
- Parallel-In Parallel-Out (PIPO): Parallel input, parallel output.

FINITE STATE MACHINE (FSM):

An FSM is a mathematical model that can be in one of a finite number of states.

FSM Design:

1. Mealy Finite State Machine:

- Outputs depend on current state and inputs.
- Output logic is combinational (no memory).
- Typically used for Control circuits and Digital systems with complex input-output relationships.

2. Moore Finite State Machine:

- Outputs depend only on current state.
- Output logic is sequential (uses memory).
- Typically used for Sequential circuits and Digital systems with simple input-output relationships.

State Diagrams:

- Graphical representation of FSM states and transitions.
- Used to design and analyze FSM behavior.

Verilog Implementation:

- Use always blocks and edge-sensitive logic.
- Define states using enumerations (enum) or parameters.

- Implement state transitions using case statements.

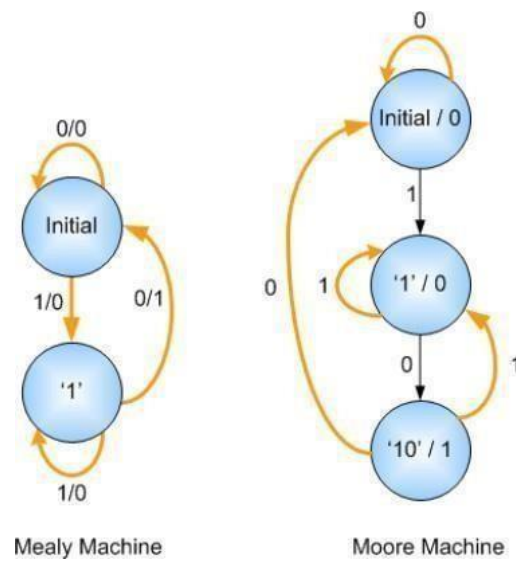


Fig. 8: Mealy and Moore State Diagrams for '10' Sequence Detector

CHAPTER-3

IMPLEMENTATION

INTRODUCTION:

PARALLEL COMMUNICATION :

In olden days we use parallel communication. It means we transmit the data bits in parallel way. At a time we can transmit more data. The data speed also high but there are some disadvantages they are due parallel transmission we use parallel lines to transmit the data from one end to another. For n no of data we use n no of paths.

SERIAL COMMUNICATION :

Serial communication means transfer the data in serially. It require only one or two paths to transfer the data due to that there is no EMI will occurred and the cost also reduce because of using low no.of paths. But When compare to parallel communication the speed of the serial communication is very low. For avoiding this drawback and increase the highspeed communication we use a circuit design called SERDES(Serializer and Deserializer).

SERDES means serializer and Deserializer. It convert parallel data into serial data to transfer the data at the transmitter side it called as a Serializer. And it converts serial data into parallel data for final output at the recivier side know as a Deserializer. SERDES have high speed communication strength to transfer the data. the speed depend upon the clock frequency, jitter, latency and line encoding technique. The circuit was desgin based on the line encoding techniques.

Various types of Line Encoding Techniques : There many types techniques are there, they are

- i) NRZ (NON RETURN TO ZERO)
- ii) MANCHESTER ENCODING
- iii) 8b/10b ENCODING
- iv) 64b/66b ENCODING
- v) 128b/130b ENCODING
- vi) PAM - 4 (PULSE AMPLITUDE MODULE)

i) NRZ (NON RETURN TO ZERO):

It means it didn't have any reset or interval between the data bits it have constant flow upto the bit period. This line encoding is simple to construct and the efficiency also good but the main issue is Synchronous problem due to constant signal recevier didnt synchronized with the transmitter clock.

ii) MANCHESTER ENCODING:

It is known as a self synchronizing signal that means single bit represent in two forms. Like if the bit

is '0' then the transition from 1 to 0 in that bit period and if the bit is '1' then the transition from 0 to 1 in that bit period. Due double transition in a bit so it require double bandwidth then the efficiency give 50% only.

iiii) **8b/10b ENCODING:**

In this encoding technique we require encoder ,decoder and for serial communication we want piso and sipo.By using encoder we encode the real 8bit data into 10b data and the 10bit data given to the decoder if encoded given to the input then it will show the real data bit in output or otherwise it shows default data. The efficiency of this technique is 80%.

iv) **64b/66b ENCODING:**

This technique also same as like as a 8b/10b encoding. But the bits are different. In this line encoding we use 64bit and it convert into 66bit . Due to this conversation the efficiency will improve compare to previous line encoding technique. The efficiency is 96.97%.it maintain good Synchronization and dc balance.

v) **128b/130b ENCODING:**

This technique also same as like as previous 8b/10b line encoding and 64b/66b line encoding. But the bits are different. In this line encoding we use 128bit and it convert into 130bit .the efficiency of this technique is 98.46%.but the main problem is construction is complex.

vi) **PAM - 4 (PULSE AMPLITUDE MODULE):**

It transmit a double data in a single bandwidth and the efficiency is also high when compare to other line encoding techniques. PAM efficiency is 200%.but there are drawbacks like increasing noise signals and increasing signal integrity.

Algorithm for SERDES Communication Protocol:

STEP 1: Start the system and initialize all registers, counters, and control signals to zero.

STEP 2: Take an 8-bit input data (data_in) from the transmitter side.

STEP 3: In the Serializer section:

- Convert the 8-bit input data into a 10-bit encoded format (en_code) using a predefined mapping pattern.
- On every positive clock edge:
 - Transmit one bit at a time from the encoded 10-bit data through the serial output line (serial_out).
 - Increment the bit counter until all 10 bits are sent.
- After all bits are transmitted, reset the bit counter and prepare for the next data.

STEP 4: The transmitted serial data is fed to the Deserializer section through a serial communication line.

STEP 5: In the Deserializer section:

- On every positive clock edge, shift each incoming serial bit into a 10-bit shift register.
- When 10 bits are completely received, transfer them to a parallel register (parallel_data).
- Decode the 10-bit data back into its original 8-bit format (data_out) using the same encoding logic as the transmitter.

STEP 6: The recovered 8-bit parallel data is then sent to the Display module for verification.

STEP 7: In the Display section:

- Generate a low-frequency clock (1 kHz) for multiplexing.
- Sequentially select each 7-segment display to show the corresponding bit of the received 8-bit data.
- Convert logic '0' and '1' into their respective 7-segment codes.
- Continuously refresh the displays to show all 8 bits simultaneously.

STEP 8: Repeat the process for every new 8-bit data input.

STEP 9: Stop the operation when data transmission is complete.

INPUT AND OUTPUT PORTS:

clk	IN			E3	▼	✓	35	LVC MOS33*	▼	3.300		
s_out	OUT			V11	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
an[7]	OUT			U13	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
an[6]	OUT			K2	▼	✓	35	LVC MOS33*	▼	3.300	12	▼
an[5]	OUT			T14	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
an[4]	OUT			P14	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
an[3]	OUT			J14	▼	✓	15	LVC MOS33*	▼	3.300	12	▼
an[2]	OUT			T9	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
an[1]	OUT			J18	▼	✓	15	LVC MOS33*	▼	3.300	12	▼
an[0]	OUT			J17	▼	✓	15	LVC MOS33*	▼	3.300	12	▼
ca[7]	OUT			H15	▼	✓	15	LVC MOS33*	▼	3.300	12	▼
ca[6]	OUT			L18	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
ca[5]	OUT			T11	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
ca[4]	OUT			P15	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
ca[3]	OUT			K13	▼	✓	15	LVC MOS33*	▼	3.300	12	▼
ca[2]	OUT			K16	▼	✓	15	LVC MOS33*	▼	3.300	12	▼
ca[1]	OUT			R10	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
ca[0]	OUT			T10	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
rx_out[7]	OUT			U16	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
rx_out[6]	OUT			U17	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
rx_out[5]	OUT			V17	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
rx_out[4]	OUT			R18	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
rx_out[3]	OUT			N14	▼	✓	14	LVC MOS33*	▼	3.300	12	▼
rx_out[2]	OUT			J13	▼	✓	15	LVC MOS33*	▼	3.300	12	▼
rx_out[1]	OUT			K15	▼	✓	15	LVC MOS33*	▼	3.300	12	▼
rx_out[0]	OUT			H17	▼	✓	15	LVC MOS33*	▼	3.300	12	▼
tx_in[7]	IN			R13	▼	✓	14	LVC MOS33*	▼	3.300		
tx_in[6]	IN			U18	▼	✓	14	LVC MOS33*	▼	3.300		
tx_in[5]	IN			T18	▼	✓	14	LVC MOS33*	▼	3.300		
tx_in[4]	IN			R17	▼	✓	14	LVC MOS33*	▼	3.300		
tx_in[3]	IN			R15	▼	✓	14	LVC MOS33*	▼	3.300		
tx_in[2]	IN			M13	▼	✓	14	LVC MOS33*	▼	3.300		
tx_in[1]	IN			L16	▼	✓	14	LVC MOS33*	▼	3.300		
tx_in[0]	IN			J15	▼	✓	15	LVC MOS33*	▼	3.300		

Fig. 1: Input and Output Ports

SIMULATION:

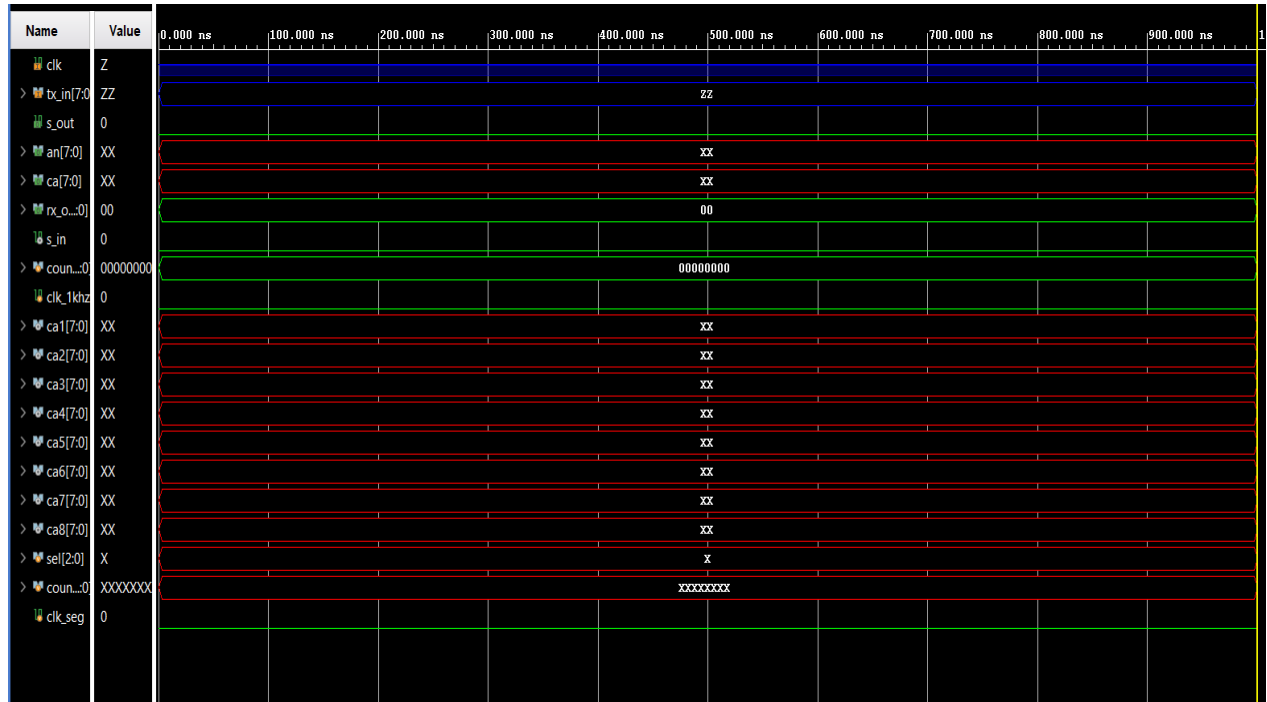


Fig. 2: Simulation

RTL SCHEMATIC:

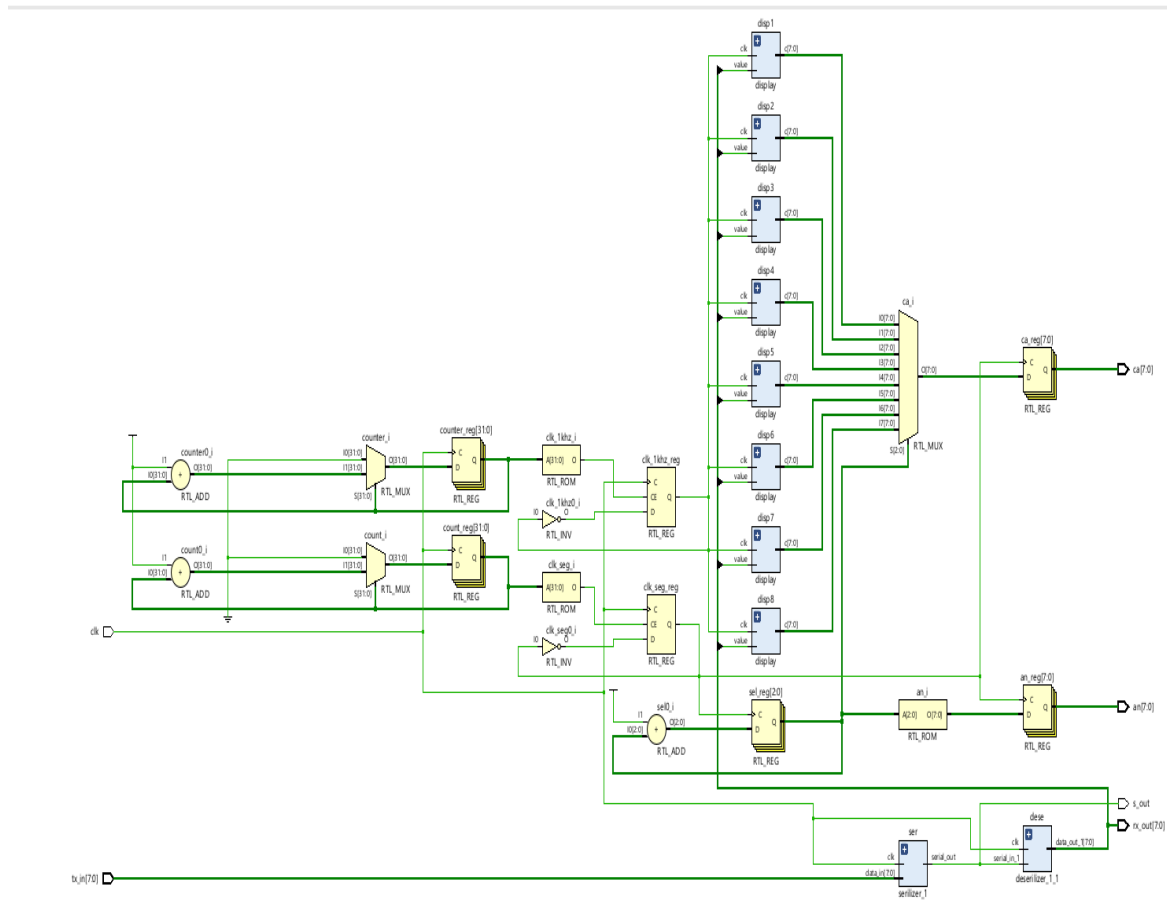


Fig. 3: RTL Schematic

SYNTHESIS:

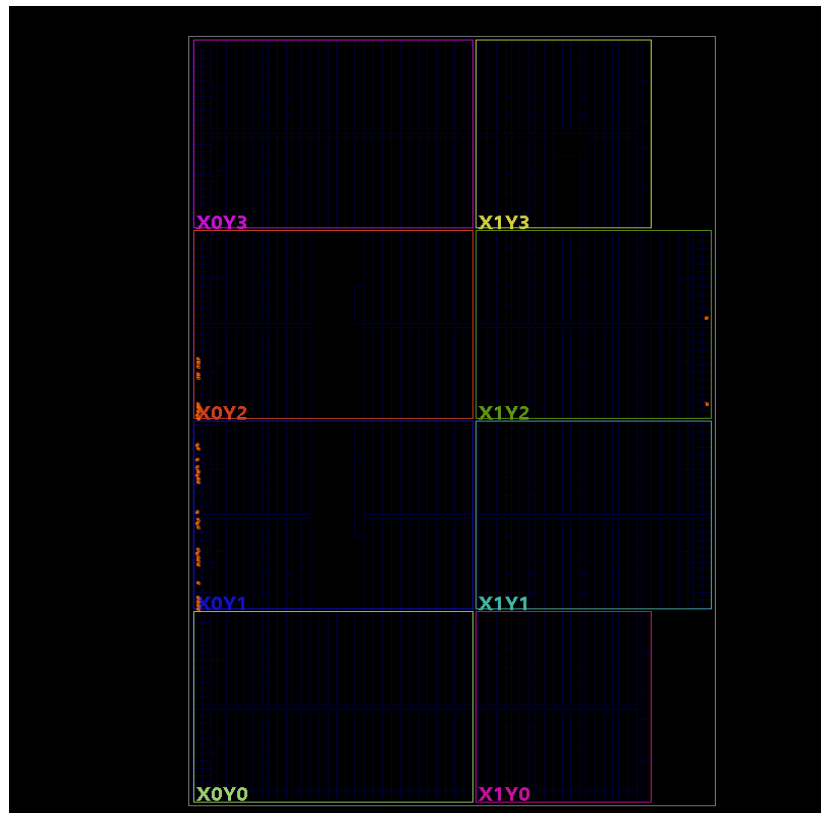


Fig. 4: Synthesis

OUTPUT ON BOARD:

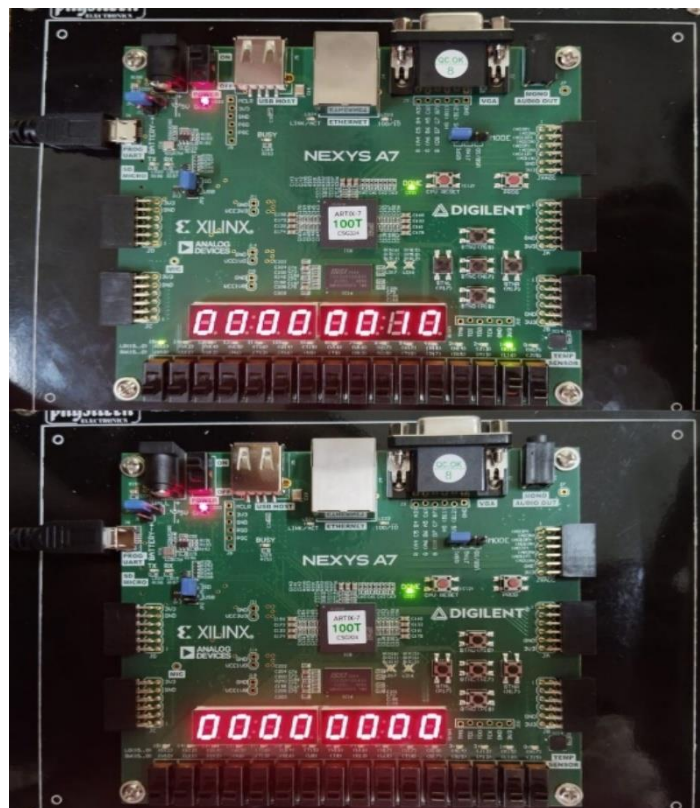


Fig. 5: Output

RESULT

The SERDES communication protocol was successfully implemented and analyzed. The system effectively converted parallel data into serial data for high-speed transmission and then accurately reconstructed it back into parallel form at the receiver end. The simulation and hardware results demonstrated reliable data transfer with minimal latency and error rate. The SERDES link achieved efficient bandwidth utilization and maintained signal integrity even at higher data rates.

CONCLUSION:

The SERDES communication protocol proves to be an efficient and reliable solution for high – speed data transmission between digital systems. By reducing the number of physical data lines and optimization bandwidth usage, SERDES enhances overall system performance while minimizing power consumption and electromagnetic interface (EMI). It is widely applicable in modern high – speed interfaces such as PCI Express, Ethernet, and optical communication systems. Over all the implementation validates that SERDES technology is essential for achieving fast, compact, and power – efficient communication in today’s VLSI and FPGA – based designs.

FUTURE SCOPE:

I plan to use the skills in future projects and at an industry level gaining here to pursue an edge in core sector after this semester. This experience has made me confident that I can perform, analyse and complete to specialize in the code field role in my future career.

REFERENCES:

1. J. L. Boswell et al., "Robust Adaptive Image Coding for Frame Memory Reduction in LCD Overdrive," IEEE Trans. Consum. Electron., vol. 59, no. 2, pp. 317-325, June 2013.
2. A. M. Nasir et al., "Design of 8T ROM embedded SRAM using double word line for low power applications," IEEE Trans. Circuits Syst. II, Express Briefs, vol. 64, no. 12, pp. 1391-1395, Dec. 2017.
3. H. Li et al., "A Hybrid SRAM/RRAM In-Memory Computing Architecture Based on a Reconfigurable SRAM Sense Amplifier," IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 31, no. 11, pp. 2102-2115, Nov. 2023.
4. Hemant Kausha & Tushar Puri – "Design of Synthesizable Asynchronous FIFO And Implementation on FPGA"- paper describing an asynchronous FIFO design (gray-code pointers, synchronizers) implemented in Verilog and synthesized on FPGA. Good for learning safe async FIFO techniques you can hook to a display controller.
5. "Verilog code for FIFO memory" – FPGA4 student (tutorial / project page) – clear Verilog FIFO implementation with testbench and usage notes (useful as a starting FIFO module you can connect to a seven – segment driver).
6. Low-Latency UART Communication Interface Implemented on Basys3" (UCRT paper) a recent project paper showing a UART system using FIFO buffers to feed LEDs and a 7-segment display. useful as a full example (FIFO peripheral →→ 7-segment).
7. Fiddling around with a FIFO circuit FPGA Coding practical explanation of circular-buffer FIFOs, read/write pointers and integration tips helpful when you want to pipe FIFO output to a display refresh module.
8. Scrolling or Moving Text Using 7-Segment LEDs in Verilog"- SimpleFPGA blog-shows driving multiple 7-segment digits with shifting/scrolling text. Combine with a FIFO that stores the characters to scroll.
9. Binary to 7-segment LED converter - Verilog" -compact decoder module (binary/BCD-7-segment) you'll need as the display front-end for FIFO output values.
10. GitHub-"circular-queue-verilog" (example repo implementing a hardware circular queue/FIFO) -an open repo with FIFO/circular-buffer Verilog you can fork and connect to a seven-segment driver in an FPGA project.